
SERA: Self Edit Robotic Adaptation

Arman Keshavarz*

Courant Institute of Mathematical Sciences
New York University

Alex Lee

Courant Institute of Mathematical Sciences
New York University

Abstract

We present a simple framework for improving robotic manipulation policies by editing their hyperparameters in a reward driven way. We start by collecting expert and random data, train a proxy reward model on actions, timesteps and goals and use it to spot episodes where the expert fails. From a baseline policy, SEAL training then generates synthetic trajectories for those failed episodes and uses a small encoder decoder to propose self edits. These hyperparameters are applied directly to the policy and the self edit module is trained to maximize predicted reward. A context summarizer gives each episode its own history aware edits which makes the updates more targeted. Over time the student model learns to output better hyperparameter edits without needing extra environment interaction. While our current setup is built for pick and place we think that self editing works reliably and can boost performance beyond the expert. This suggests that reward guided hyperparameter generation is a promising direction for adaptable robotic policies.

1 Introduction

Robot reinforcement learning policies usually break in surprisingly simple ways. A policy that works perfectly in most situations will suddenly fail on one odd episode because the gripper hesitated, the timing was slightly off or a small hyperparameter was not tuned quite right for that specific setup. In reality a lot of these failures are just the kind of little mistakes in the training data that never get fixed because collecting new data is costly and tuning hyperparameters by hand is slow.

Many of these failures are fixable if the policy could just tweak a few of its own internal settings. Humans do this constantly. If we miss a basketball shot to the left we adjust our aim to the right the next time we shoot. If we burn the eggs while cooking breakfast we lower the heat the next time. If we fail on a bench press rep we lower the weight for the next set. We are constantly learning and updating our mental training set. Policies on the other hand typically do not adjust anything once training is over. They inherit a fixed set of hyperparameters and are stuck with them forever.

This work explores that idea. Instead of retraining the whole policy or collecting more demos we ask if we can let the policy modify the hyperparameters that shape its own behavior. Rather than treating hyperparameters as constants we treat them as something the model can change when an episode goes wrong.

To make this possible without more environment interaction we build a reward model from expert and random data. That reward model becomes the critic that tells us whether a change in hyperparameters helped or made things worse. When the expert fails on a particular episode we use SEAL training to generate synthetic rollouts for that failure and search for a small hyperparameter edit that would have improved it. A small neural network style self edit module proposes these edits. It is conditioned on everything that happened in the episode and learns to make better edits over time by chasing higher predicted reward.

*Corresponding author.

This approach ends up giving the policy a way to adjust itself and control how it behaves in order to optimize rewards. Once again the current system is tailored to pick and place because the hyperparameters were designed for that domain. However the proof of concept works. The policy can patch its own behavior without new rollouts and can outperform the original expert on cases where the expert struggled.

2 Related Works

2.1 Policy optimization in RL

Classic RL algorithms like PPO focus on improving a policy by directly adjusting its parameters through interaction with the environment. PPO uses a clipped surrogate objective that lets the policy take multiple optimization steps on the same batch of data while keeping updates conservative. This prevents the destructive jumps that plain policy gradients often produce. By alternating between rollout collection and several epochs of first order optimization PPO strikes a practical balance between stability and sample efficiency. But critically PPO and similar methods optimize the policy weights themselves. Hyperparameters remain fixed and improvement depends on collecting fresh environment data.

2.2 Self editing and hyperparameter level adaptation

Zweiger et al. [1] introduced Self Adapting Language Models (SEAL). This is a framework that enables models to autonomously direct their own adaptation process by generating synthetic data or optimization directives which are contextualized to new inputs. The approach operates as a bi level optimization problem. In the inner loop the model generates a self edit such as rephrasing a passage or selecting hyperparameters like learning rate. It immediately updates its weights via supervised finetuning (SFT) on this generated content. In the outer loop the self edit generation policy is optimized using reinforcement learning. They specifically used ReSTEM where the reward is derived from the updated model’s performance on a held out downstream task. By treating the adaptation strategy itself as a learnable policy rather than a fixed heuristic SEAL demonstrates improved performance in knowledge incorporation and few shot reasoning tasks compared to standard in context learning.

Our work builds on this SEAL viewpoint but adapts it to robotic manipulation where failures often stem from small timing or control level mismatches. These are easier to fix with hyperparameter changes than full policy retraining. Instead of using SEAL to rewrite language model internals we use a lightweight self edit module to generate hyperparameter edits for a robotic policy. And unlike PPO our approach uses a learned reward model to evaluate edits offline. This lets the policy improve even on failure episodes where expert demonstrations break down.

2.3 Adaptable Robot Movements

Song et al. [2] addressed the challenge of transferring locomotion policies to physical robots with unknown dynamics by employing an evolutionary meta learning framework. Their method trains a meta policy in simulation using Model Agnostic Meta Learning (MAML). It explicitly optimizes the network initialization for malleability rather than immediate performance. During deployment they utilize an Evolution Strategy (ES) to rapidly fine tune the policy weights on the physical hardware. This allows a legged robot to adapt to unseen perturbations within a handful of trials. While this approach shares our goal of solving the sim to real gap via rapid autonomous adaptation it operates by directly perturbing the policy parameters (θ) in the outer loop. In contrast our work creates a higher level abstraction for adaptation. Instead of modifying weights directly our meta learner modulates the learning conditions via synthetic data generation and hyperparameter control. This allows the base policy to update itself using standard gradient based supervision.

3 Methodology

Our goal with SEAL is to build a robot that does not just memorize a task but realizes when it is failing and tweaks its own learning strategy to fix it. Instead of relying on a human engineer to

manually tune hyperparameters when the robot gets stuck we train a neural network to do that tuning on the fly. Our pipeline consists of four main phases: collecting the data, building the critic (Proxy Reward), building the editor (The Generator) and running the adaptation loop.

3.1 Data collection

First we needed to teach it what success and failure actually look like. To generate successes we wrote a scripted policy that uses hardcoded phases (move to object, grasp, lift) to solve the FetchPickAndPlace task. We generated 5000 successful trajectories this way. To show the robot what not to do (generating failures) we ran a policy that takes random actions. We collected 5000 failed trajectories where the robot flails around, jitters and just misses the object. These datasets formed the baseline ground truth for our reward model.

3.2 Proxy Reward Model

Just like in the original SEAL paper we wanted our model to have an internal critic of sorts that would be able to predict whether a trajectory would be successful or not. So we trained a “proxy reward model” to replace the ground truth evaluator typically used in these reinforcement learning tasks. The way the model works is as follows:

- **Input:** Takes a snapshot of the current context which includes the observation, the goal, the action taken and the timestep.
- **Architecture:** A simple 3 layer MLP that we were able to train on our Intel CPUs.

$$\text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear}$$

- **Output:** Learns to output a number as close to one as possible for expert trajectories (successes) and as close to zero as possible for random trajectories (failures).

After training this model acts as a judge to predict whether an action would be a move that results in success or not.

3.3 The Self Edit Generator and Inner Loop

We built an encoder decoder style network that acts as our editor.

The Encoder This is a Multi Layer Perceptron (MLP) that maps the high dimensional context vector to a latent distribution z .

- **Input:** Context dimension.
- **Output:** Parameters μ and $\log \sigma$ for a diagonal Gaussian distribution.
- **Sampling:** We sample a latent edit vector $z \in \mathbb{R}^{16}$ using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon$$

where ϵ follows a standard normal distribution. This stochasticity allows the generator to explore different “editing strategies” for the same failure mode.

The Decoder This is a second MLP that maps the latent code z to a vector of continuous coefficients $h \in \mathbb{R}^8$.

- **Activation:** The final layer uses a Tanh activation to bound all outputs to the range $[-1, 1]$.
- **Scaling:** These raw values are linearly projected into meaningful ranges. For example we use meters for positions and scalars for velocity.

3.3.1 Action Space

The decoder outputs 8 scalars designed to address failures in our robot simulation of pick and place.

- **Smooth Factor:** Controls kernel size of Gaussian filter applied to action sequence to reduce jitter.
- **Action Scale:** Scalar for the velocity command. This lets robot move more aggressively or cautiously.
- **Approach Height:** Vertical bias.
- **Grasp Offset:** Separate for both X and Y directions. This is an offset added strictly during the grasping phase to address misalignments that happen.
- **Pregrasp Lift:** Bias before gripper closes to prevent robot from knocking the object over. This happened a few times in our training.
- **Postgrasp Lift:** Upward bias immediately after gripping so it can actually move it properly.
- **Gripper Bias:** Adjusting the sensitivity of the gripping and letting go.

3.3.2 The Editing Function

We do not just pass the new hyperparameters as inputs to the policy. We use them to synthetically rewrite the history of the failed trajectory. Given a failed trajectory $T(s, a)$ we generate a new synthetic Trajectory $T'(s, a')$ by applying the transformation function at every timestep.

$$a' = \text{smooth}(a_t, \alpha) \cdot v + \text{Bias}(s_t, h_{\text{app}}, s_{xy}, l)$$

Additionally the bias terms are state dependent. For example the approach height bias is only added if the robot's z coordinate exceeds a threshold and the postgrasp lift is only triggered if the gripper state indicates "closed." This allows the Generator to perform thought out edits (e.g. "only lift higher when you are near the table") rather than applying a global offset that would ruin the rest of the motion. This modified dataset T is then treated as the expert data for the purpose of the inner loop update. We effectively trick the student policy into imitating a trajectory that never happened but would have succeeded if it had.

3.3.3 Inner Loop (Total)

As we explained before the generator reviews the failure and proposes a set of "Edit Parameters." Then we take the failed trajectory and mathematically modify the actions using those parameters. For example we overwrite the old clumsy actions with new smoother ones defined by the edit. Finally we take a base "Student" policy (a standard Behavior Cloning network) and train it for a few quick steps on this new synthetic successful data. This results in a temporary specialized brain that thinks it knows how to solve the specific problem it just failed at.

3.4 Meta Learning

Lastly we have to teach the Generator how to make good edits. We do this using REINFORCE which is our policy gradient method.

1. Let the generator propose edits.
2. Run the Inner Loop to train the student.
3. Rollout the student in the environment.
4. Score the student's performance using the reward model.
5. If the student succeeds the generator gets a positive reward. If the student fails the generator is penalized.

Over time the generator learns good mappings. For example when the block is far away it suggests a higher action scale. When the block is slippery it suggests a tighter grip.

4 Experimental Setting

4.1 Environment and Task

We evaluate all methods on FetchPickAndPlace. This is a standard continuous control robotic manipulation task with sparse success signals. The task requires a robotic arm to grasp an object

and place it at a target location. This demands precise coordination and robustness to execution errors. All experiments are conducted in a fixed simulator configuration to ensure controlled and fair comparisons across adaptation methods.

To isolate the effects of adaptation the simulator is reset to the exact same initial state at the start of every evaluation episode. This eliminates variability due to environment stochasticity and ensures that performance differences arise solely from the adaptation mechanisms.

4.2 Demonstration Dataset and Base Policy

A fixed dataset of expert demonstration trajectories is used throughout all experiments. From this dataset we train a base policy using standard behavior cloning (BC) in an offline setting. The base policy serves as the initialization point for all adaptation methods.

The BC policy is trained to convergence using mean squared error on continuous actions. During evaluation the base policy is frozen and reused across all methods to ensure identical initial conditions.

In addition we train a proxy reward model on demonstration data. This reward model outputs a dense success score at each timestep which enables fine grained evaluation and adaptation even in the presence of sparse environment rewards. All methods are evaluated using the same reward model.

4.3 Evaluation Protocol

For each method we evaluate performance over 50 episodes. Each is initialized from the same simulator state and uses the same observation normalization, policy architecture and reward model. No online environment interaction beyond these evaluation rollouts is permitted during adaptation.

Performance is measured using:

- **Total Proxy Reward:** Computed as the sum of reward model outputs over a rollout.
- **Distance Error:** Defined as the final object to goal Euclidean distance.

We report improvements relative to the base policy by measuring the percentage increase in total proxy reward and the percentage decrease in distance error.

4.4 Baselines

We compare against several representative adaptation strategies:

- **Base Policy (BC / ICL):** A behavior cloned policy trained offline from demonstrations with no test time adaptation.
- **Test Time Training (TTT):** The student policy is initialized as a copy of the base policy and adapted at test time by running gradient steps on the BC loss using the fixed demonstration dataset.
- **TTT + Self Edit:** A fixed hand designed edit is applied to the failed episode trajectory before performing TTT. This edit encodes heuristic corrections and is identical across episodes.
- **CEM Optimized Self Edit:** Instead of a fixed edit a latent edit vector is optimized using the Cross Entropy Method (CEM). The latent distribution is iteratively updated to maximize proxy reward and the resulting edit is applied before TTT adaptation.

4.5 Proposed Method

Structured Editing with Meta Learned Generator (Ours): Our method introduces a structured edit distribution parameterized by a meta learned generator. Given a failed trajectory the generator outputs a distribution over structured edit parameters which are decoded into trajectory level edits. The edited trajectory is then used to adapt the student policy via behavior cloning. Unlike heuristic or search based baselines the generator is trained across episodes to produce edits that consistently improve downstream policy adaptation. During evaluation rollouts are used to update the generator closing the loop between trajectory editing and policy improvement.

5 Results

Table 1 summarizes performance across all methods.

Table 1: Performance comparison on FetchPickAndPlace (50 episodes)

Method	Reward Increase (%)	Distance Error Decrease (%)
BC / ICL	0.0	0.0
TTT	6.16	4.41
TTT + Self Edit	17.04	35.13
CEM	15.15	12.20
Ours	32.71	52.45

Our method achieves the largest improvement in total proxy reward. It more than doubles the gains of the strongest baseline. Additionally it reduces final distance error by over 52% which indicates not only higher success likelihood but also more precise task execution.

The results demonstrate that naive test time fine tuning provides limited gains while trajectory level editing significantly improves adaptation performance. However fixed or search based edits lack consistency across episodes. By contrast our meta learned edit generator captures reusable structure in successful corrections. This enables more efficient and stable test time adaptation. These findings highlight the importance of explicit trajectory structure and meta learning for scalable test time policy improvement.

6 Limitations

While our approach demonstrates substantial gains over existing test time adaptation and editing baselines it also inherits several limitations that are important to acknowledge.

Catastrophic Forgetting Our method relies on test time policy updates using edited trajectories. As a result the adapted student policy may overfit to the current episode or local context leading to catastrophic forgetting of behaviors that were previously successful under the base policy. Our method introduces additional hyperparameters beyond those required for standard behavior cloning or test time training. These include learning rates for the edit generator, the number of adaptation steps and the scale of the structured edit distribution. Ironically the performance is sensitive to these choices and suboptimal tuning can result in instability or catastrophic forgetting.

Computational Overhead Compared to simpler baselines such as TTT or fixed self editing our method incurs additional computational cost. Each evaluation episode involves sampling structured edits from a learned distribution, decoding edits into trajectory modifications, performing behavior cloning updates on the edited data and updating the edit generator based on rollout performance. This results in longer per episode runtimes than direct gradient based adaptation or heuristic edits. While our method is significantly more sample efficient than search based approaches such as CEM it remains slower than lightweight alternatives that do not involve meta learning or generator updates.

References

- [1] Zweiger, A., Pari, J., Guo, H., Akyürek, E., Kim, Y., & Agrawal, P. (2025) Self-Adapting Language Models.
- [2] Finn, C., Abbeel, P., & Levine, S. (2017) Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks.
- [3] Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. A., & Hardt, M. (2020) Test-Time Training with Self-Supervision for Generalization under Distribution Shifts.
- [4] Wang, D., Shelhamer, E., Liu, S., Olshausen, B. A., & Darrell, T. (2020) Fully Test-time Adaptation by Entropy Minimization.
- [5] Ross, S., Gordon, G. J., & Bagnell, J. A. (2010) No-Regret Reductions for Imitation Learning and Structured Prediction.

[6] Ouyang, L. et al. (2022) Training language models to follow instructions with human feedback.